

Discrete Dynamics

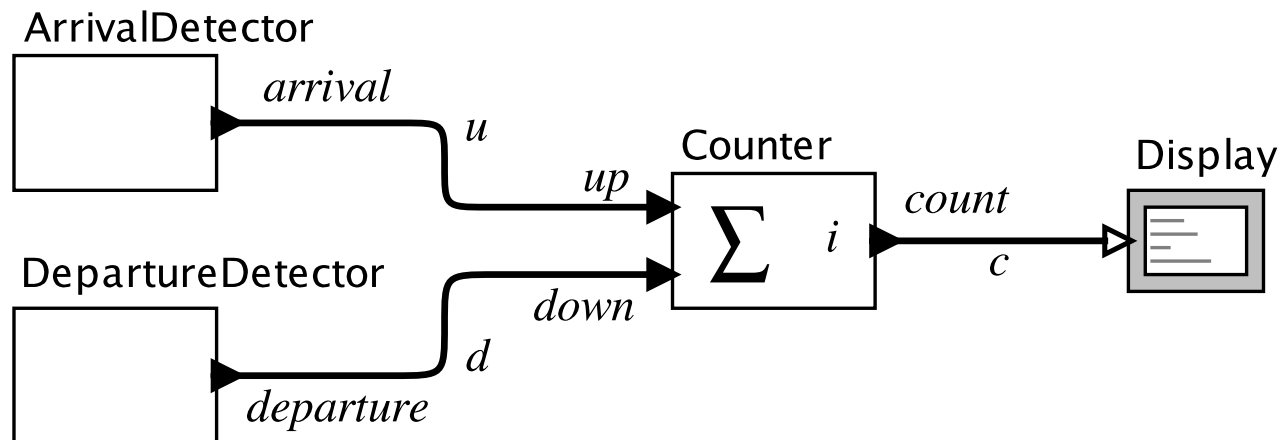
Modeling Modal Behavior

Models of embedded systems

- continuous components.
 - ✦ continuous components evolve smoothly
 - ✦ modeling by ordinary differential or integral equation
- discrete components
 - ✦ discrete components evolve abruptly
 - ✦ modeling by state machine
- hybrid system
 - ✦ discrete and continuous components
 - ✦ modeling by state machine

Discrete Systems

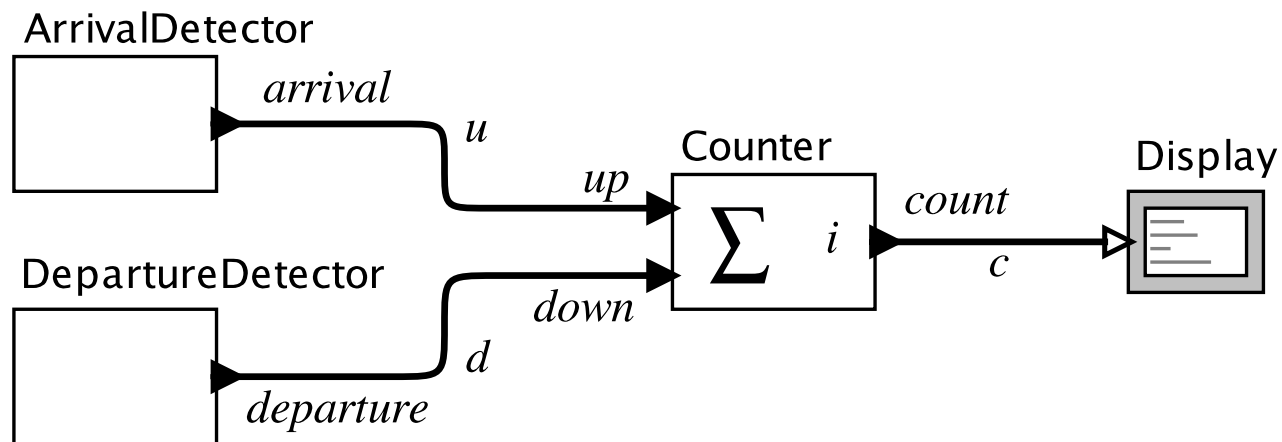
- Example: count the number of cars that enter and leave a parking garage:



- Each entry or departure is modeled as a discrete event: u and d
- A discrete event occurs at an instant of time rather than over time
- a function: $u : \mathbb{R} \rightarrow \{absent, present\}$

Pure signal

- A signal is present or absent at any given time

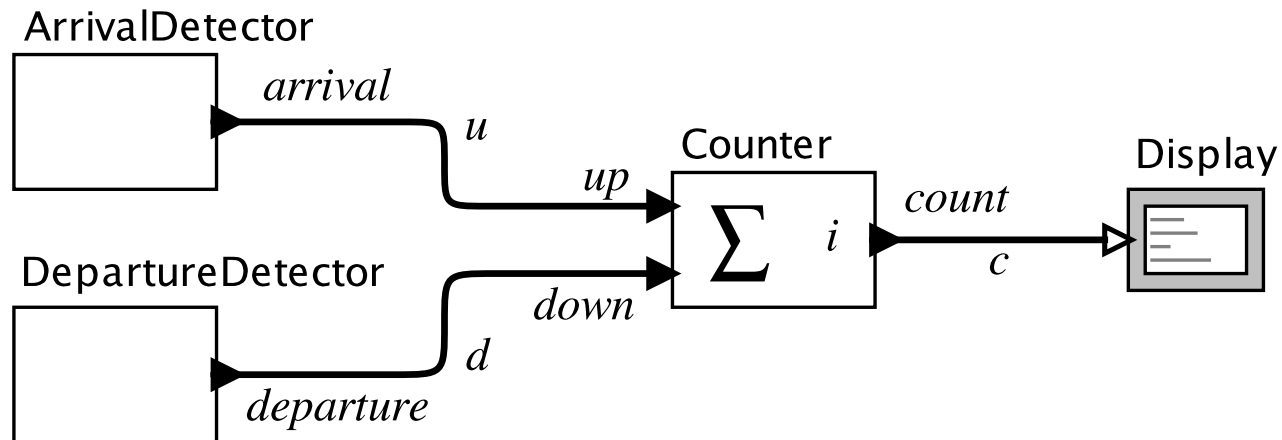


Pure signal: $u : \mathbb{R} \rightarrow \{absent, present\}$

$u(t)$ is either absent, meaning that there is no event at that time, or present, meaning that there is.

Counter

- Two input u and d ,
- one output c , count and produces on the output the new value of the count



- signal c

$$c: \mathbb{R} \rightarrow \{absent\} \cup \mathbb{Z} .$$
- c is not pure, it is either absent or present.
 when it is present, it has a value (an integer)

discrete signals

- absent at most of time, the present time can count
- each time it is present, then we have a discrete event.
- formally, let $T \subseteq \mathbb{R}$ be the set of times where e is present

$$T = \{t \in \mathbb{R} : e(t) \neq \text{absent}\}$$

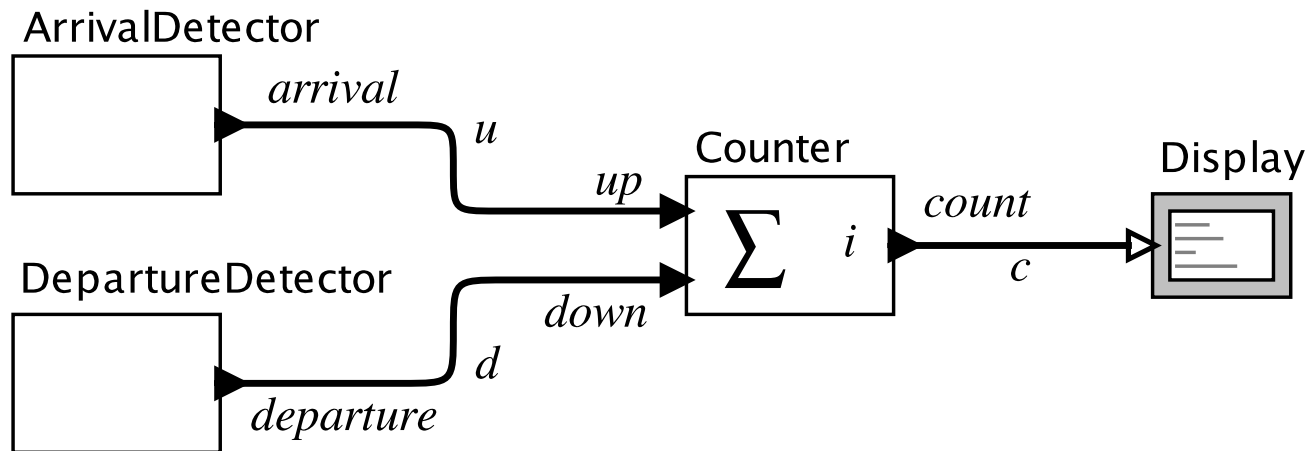
e is discrete if there exists a one-to-one function
 $f: T \rightarrow \mathbb{N}$

reaction

- For any $t \in \mathbb{R}$ where $u(t) \neq \text{absent}$ or $d(t) \neq \text{absent}$ the Counter **reacts**, and produces an output value in $\mathbb{N}$.
- The dynamics of a discrete system can be described by **reactions**
- Reactions are triggered by the environment.
 - ◆ reaction is triggered by the event triggered

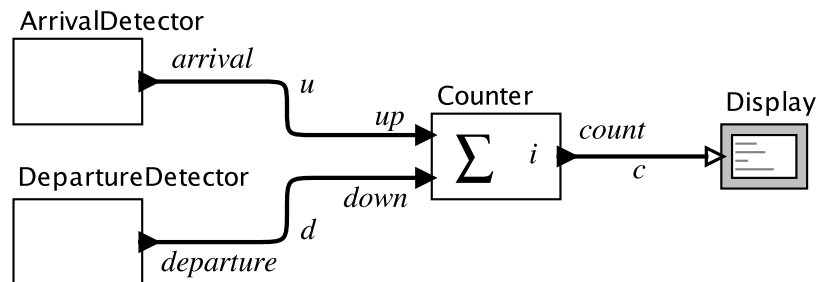
reaction(cont')

- A particular reaction will observe the values of the inputs at a particular time t and calculate output values for that same time t .



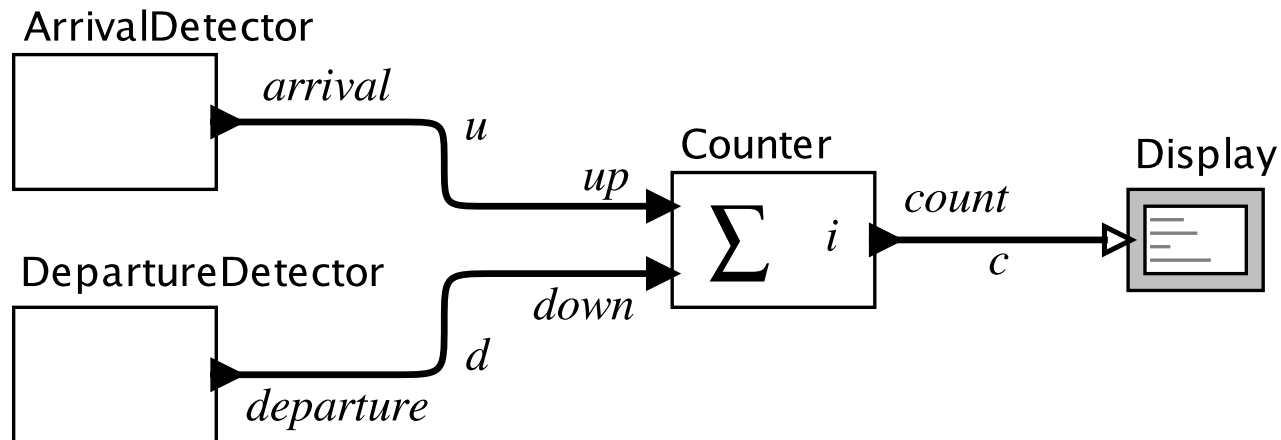
Input and Output Valuations at a Reaction

- input ports $P = \{p_1, \dots, p_N\}$, where p_i is the name of the i -th input port.
- type of port p : V_p is the values that may be received on port p when the input is present.
- A valuation of the inputs P is an assignment of value in V_p to each variable $p \in P$ or an assertion that p is absent.
- $p \in V_p \cup \{absent\}$



Input and Output Valuations at a Reaction

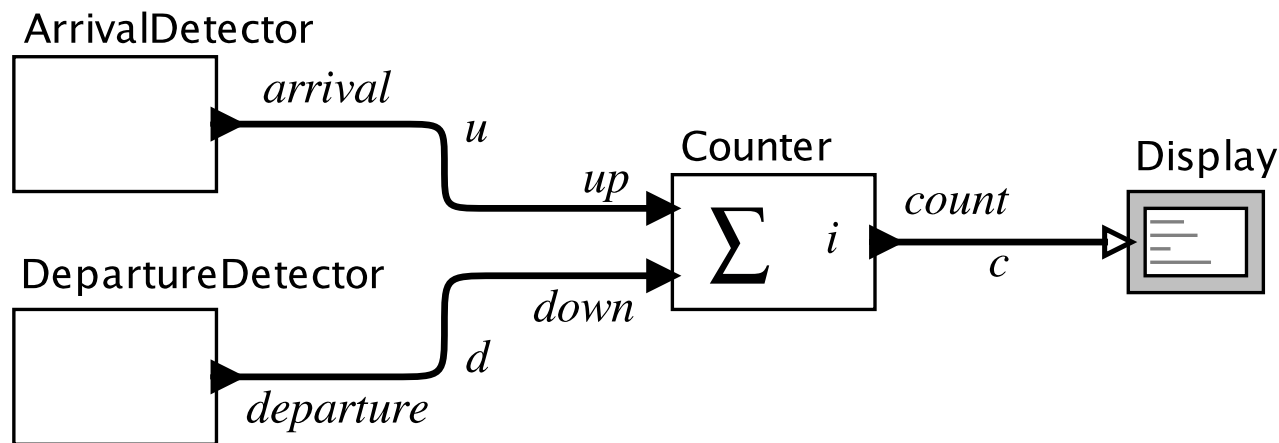
- Outputs are similarly designated. output ports $Q = \{q_1, \dots, q_M\}$ with types $V_{q1}, \dots, V_{qM}$.
- Example. For garage, $P = \{up, down\}$, $Q = \{count\}$.
- types are $V_{up} = V_{down} = \{present\}$, $V_{count} = \mathbb{Z}$



State Space

- state: its condition at a particular point in time. A summary of the past
- Discrete models with finite state spaces are called finite-state machines (FSMs)
- A practical parking garage has a finite number M of spaces. So the state space for the counter is

$$\text{States} = \{0, 1, 2, \dots, M\} .$$



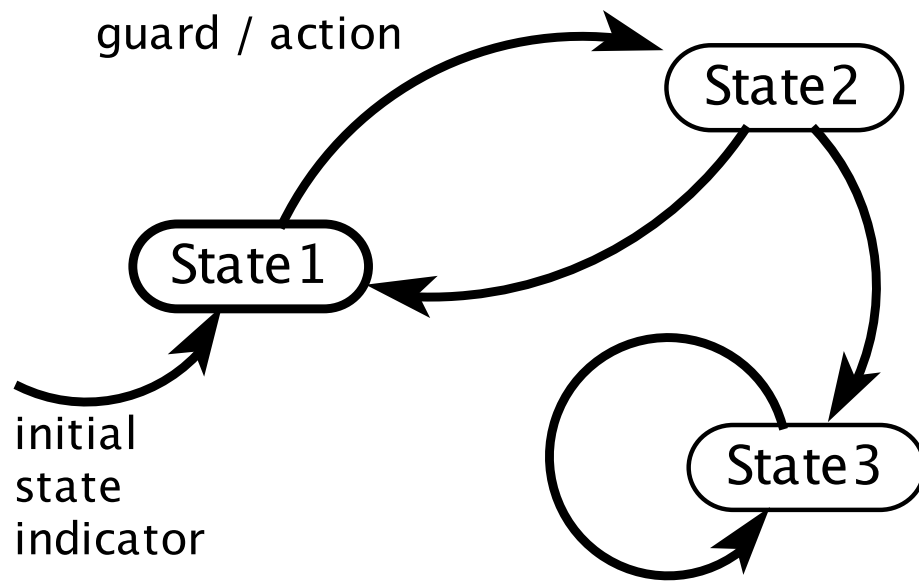
Finite-State Machines

- A state machine is a model of a system with **discrete dynamics** that at each **reaction** maps **valuations** of the inputs to valuations of the outputs
- A finite-state machine (FSM) is a state machine where the set States of possible states is finite.

Finite-State Machines

- FSM is expressed by a graph
 - ✦ state by node
 - ✦ reaction by transition
 - Transitions between states govern the discrete dynamics of the state machine and the mapping of input **valuations** to output valuations.

FSM Notation



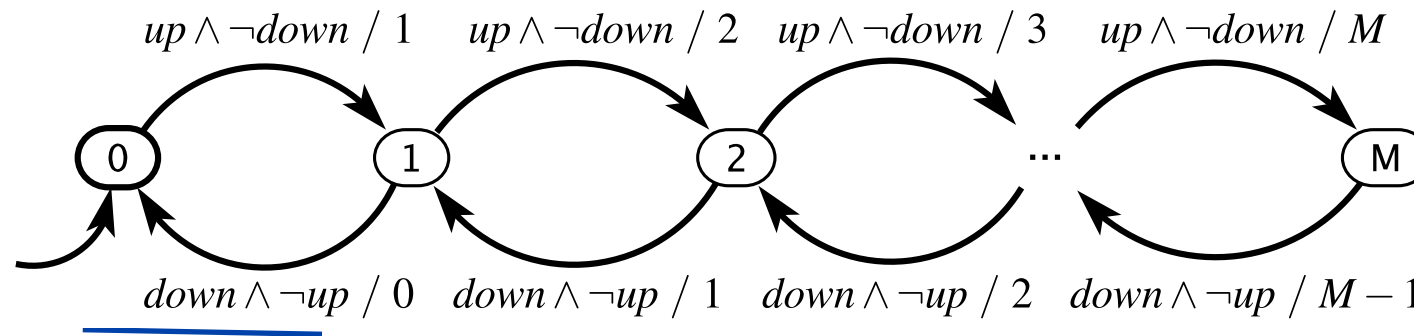
- The **guard** determines whether the transition may be taken on a reaction. It is a predicate.

- The **action** specifies what outputs are produced on each reaction.
- **self transition**.
- **initial state**: the beginning of a discrete system

Garage Counter Finite State Machine (FSM) in Pictures

inputs: $up, down$: pure

output: $count : \{0, \dots, M\}$



Guard g is specified using the predicate

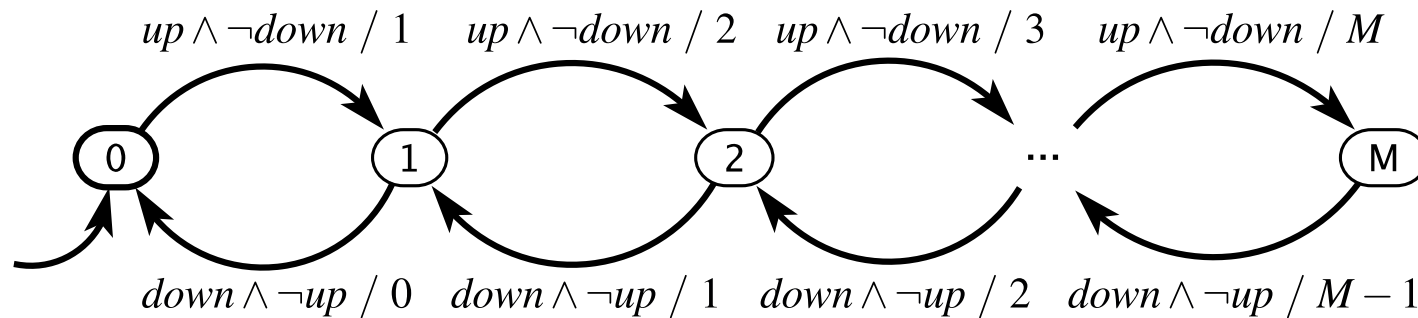
$$up \wedge \neg down$$

which means that $down$ has *absent* value and up has *present* value.

Garage Counter Finite State Machine (FSM) in Pictures

inputs: $up, down$: pure

output: $count : \{0, \dots, M\}$

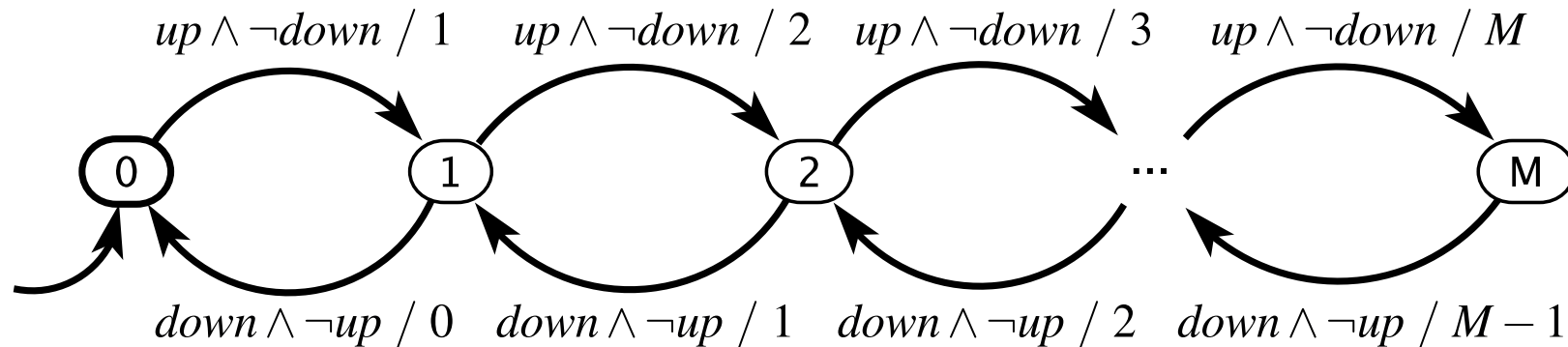


Initial state

Garage Counter Finite State Machine (FSM) in Pictures

inputs: $up, down$: pure

output: $count : \{0, \dots, M\}$



output valuation

- The output port *count* is not explicitly named because there is only one output port, and hence there is no ambiguity

Examples of Guards for Pure Signals

$true$ Transition is always enabled.

p_1 Transition is enabled if p_1 is present.

$\neg p_1$ Transition is enabled if p_1 is absent.

$p_1 \wedge p_2$ Transition is enabled if both p_1 and p_2 are present.

$p_1 \vee p_2$ Transition is enabled if either p_1 or p_2 is present.

$p_1 \wedge \neg p_2$ Transition is enabled if p_1 is present and p_2 is absent.

Examples of Guards for Signals with Numerical Values

p_3 Transition is enabled if p_3 is present (not absent).

$p_3 = 1$ Transition is enabled if p_3 is present and has value 1.

$p_3 = 1 \wedge p_1$ Transition is enabled if p_3 has value 1 and p_1 is present.

$p_3 > 5$ Transition is enabled if p_3 is present with value greater than 5.

Examples of Action for Signals with Numerical Values

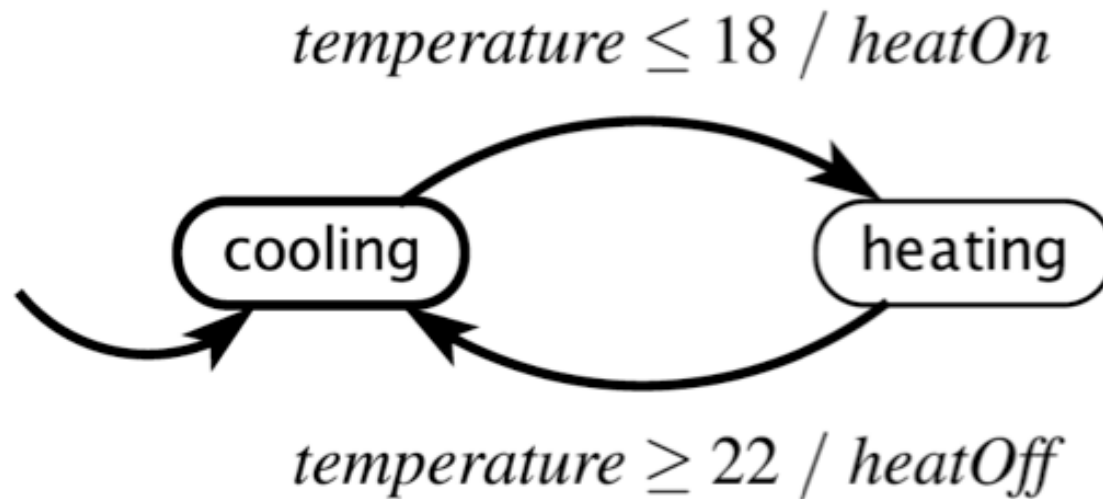
- q_1 q_1 is present and q_2 and q_3 are absent.
- q_1, q_2 q_1 and q_2 are both present and q_3 is absent.
- $q_3 := 1$ q_1 and q_2 are absent and q_3 is present with value 1.
- $q_3 := 1, q_1$ q_1 is present, q_2 is absent, and q_3 is present with value 1.
- (nothing) q_1, q_2 , and q_3 are all absent.

where q_1 and q_2 are pure outputs and q_3 has type $\mathbb{N}$

Example: Thermostat

input: *temperature* : $\mathbb{R}$

outputs: *heatOn*, *heatOff* : pure



- Setpoint: 20 degree
- guard: $temperature \leq 18$, $temperature \geq 22$
 - hysteresis
 - avoid chattering

event-triggered VS. time-triggered

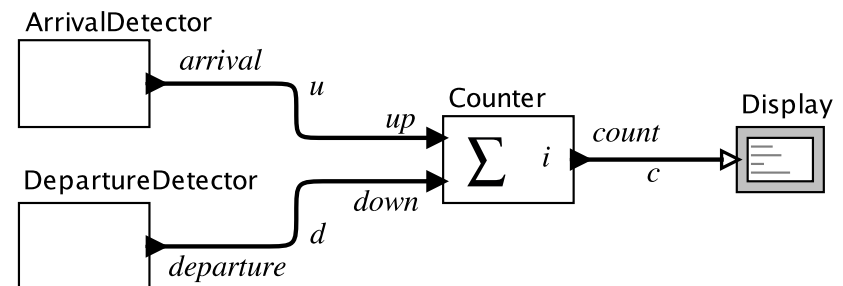
- **event triggered** it will react whenever a input is provided
 - examples above are event-triggered
- **time triggered** reacts at regular time intervals
- The definition of the FSM does not change in these two cases in this example.

when a reaction occurs

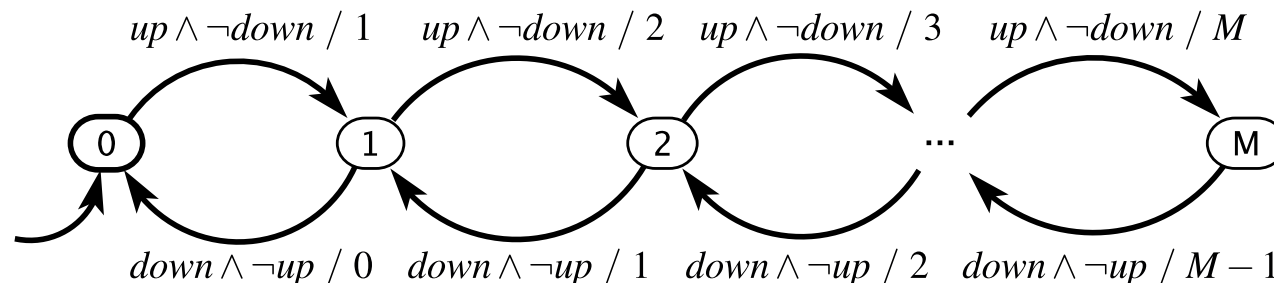
- the environment determines when the machine reacts
- If inputs have a valuation, then the FSM should react
 - ✦ assign a valuation to output ports
 - ✦ change(possibly) to a new state
 - ✦ stay in the same state if no guard on any transition is true.

all possible behavior

- FSM defines all possible behavior
 - some explicitly define
 - some implicitly define
- prepare for analysis
 - ◆ garage example



inputs: $up, down$: pure
output: $count : \{0, \dots, M\}$



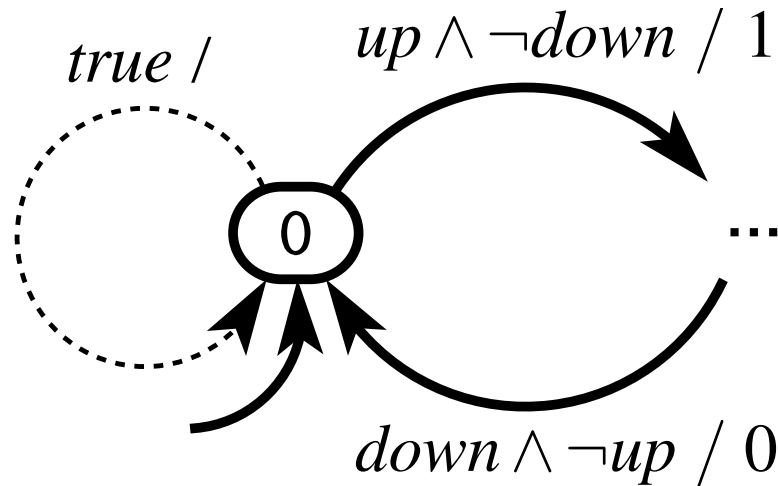
stuttering reaction (哑响应)

- A **stuttering** reaction is one where the inputs and outputs are all absent and the machine does not change state.
- example garage
 - ◆ stuttering reaction when $down=up=absent$ in state 0
 - ◆ not stuttering reaction when $down=present$ and $up=absent$ in state 0

Default Transition

- Implicitly reaction in a diagram
- for example: *down=present* in state 0
- to emphasize such reactions, using a **default transition**
- dashed line in the diagram, and a default transition is enabled if no non-default transition is enabled and if its guard evaluates to true.

Example for Default transition



- priority to transition
 - ◆ an ordinary transition is prior to a default transition

A Mathematical Notation for FSM

- a five-tuple

$(States, Inputs, Outputs, update, initialState)$,
where

$States$ is a finite set of **states**;

$Inputs$ is a set of input **valuations**;

$Outputs$ is a set of output **valuations**;

$update : States \times Inputs \rightarrow States \times Outputs$ is an **update function**, mapping a state and an input valuation to a next state and an output valuation;

$initialState$ is the **initial state**.

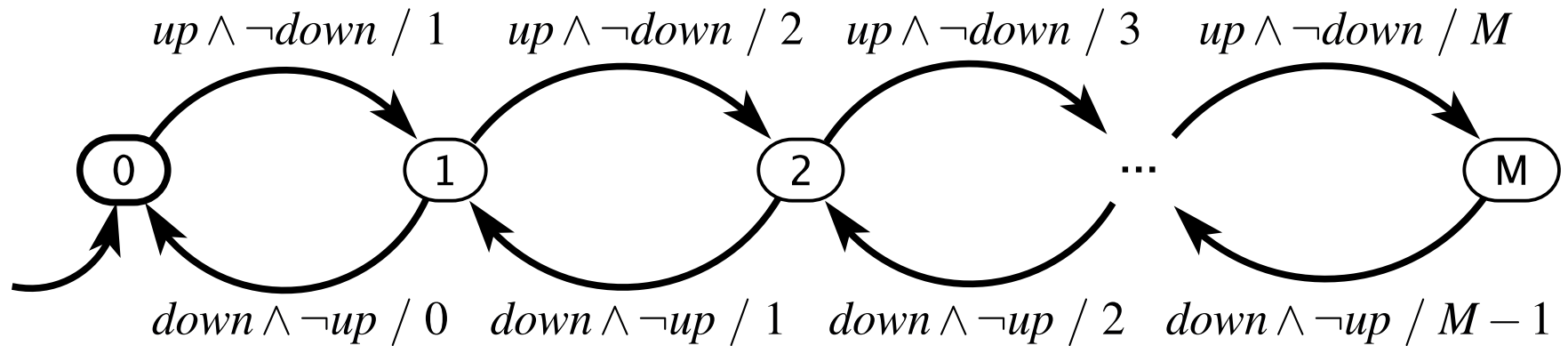
number states

- The FSM reacts in a sequence of **reactions**.
- $s : \mathbb{N} \rightarrow States$ be a function that gives the state of an FSM at reaction $n \in \mathbb{N}$.
- $s(0) = initialState$
- $x : \mathbb{N} \rightarrow Inputs$ and $y : \mathbb{N} \rightarrow Outputs$
- $x(0) \in Inputs, y(0) \in Outputs$
 $(s(n + 1), y(n)) = update(s(n), x(n))$

Garage Counter Mathematical Model

inputs: $up, down : \text{pure}$

output: $count : \{0, \dots, M\}$



$(States, Inputs, Outputs, update, initialState)$,
where

Garage Counter Mathematical Model

$$States = \{0, 1, \dots, M\}$$

$$Inputs = (\{up, down\} \rightarrow \{present, absent\})$$

$$Outputs = (\{count\} \rightarrow \{0, 1, \dots, M, absent\})$$

$$initialState = 0$$

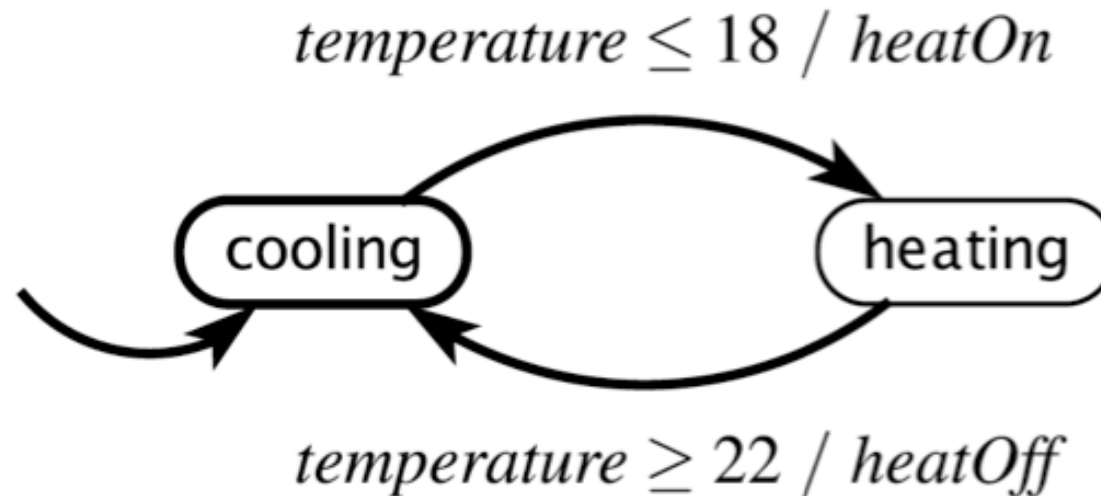
The update function is given by

$$update(s, i) = \begin{cases} (s + 1, s + 1) & \text{if } s < M \\ & \wedge i(up) = present \\ & \wedge i(down) = absent \\ (s - 1, s - 1) & \text{if } s > 0 \\ & \wedge i(up) = absent \\ & \wedge i(down) = present \\ (s, absent) & \text{otherwise} \end{cases}$$

Example: Thermostat

input: *temperature* : $\mathbb{R}$

outputs: *heatOn*, *heatOff* : pure



Exercise: From this picture, construct the formal mathematical model?

Determinacy and Receptiveness

- **Receptiveness:** For any input values, some transition is enabled for each state. Structure together with the implicit default transition ensures that FSMs are receptive.
- **Determinism:** In every state, for all input values, exactly one (possibly implicit) transition is enabled.

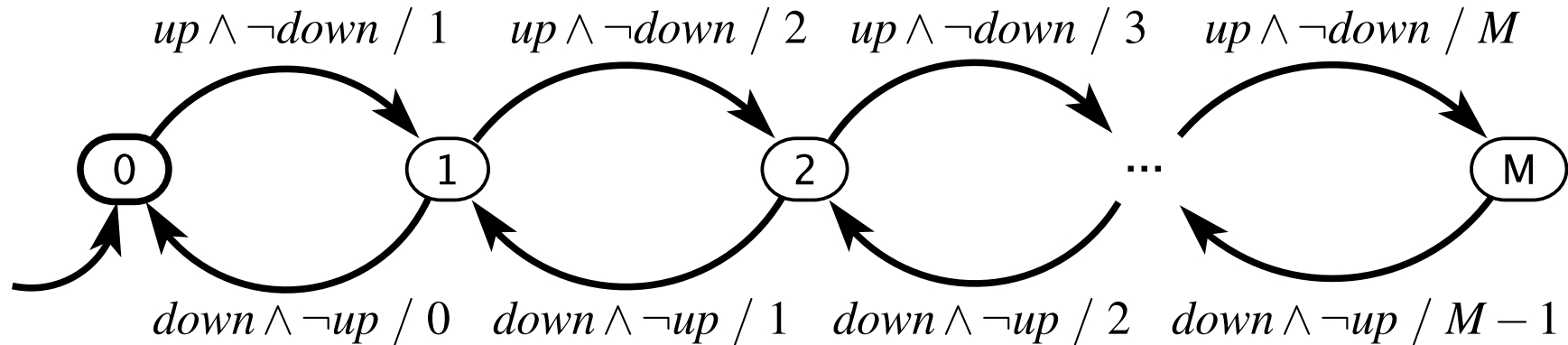
Extended State Machines

- Thinking garage, If M is large, the bubble-and-arc notation becomes uncontrollably.
- An **extended state machine** solves this problem by augmenting the FSM model with variables .
- Garage example

Extended State Machines

inputs: $up, down$: pure

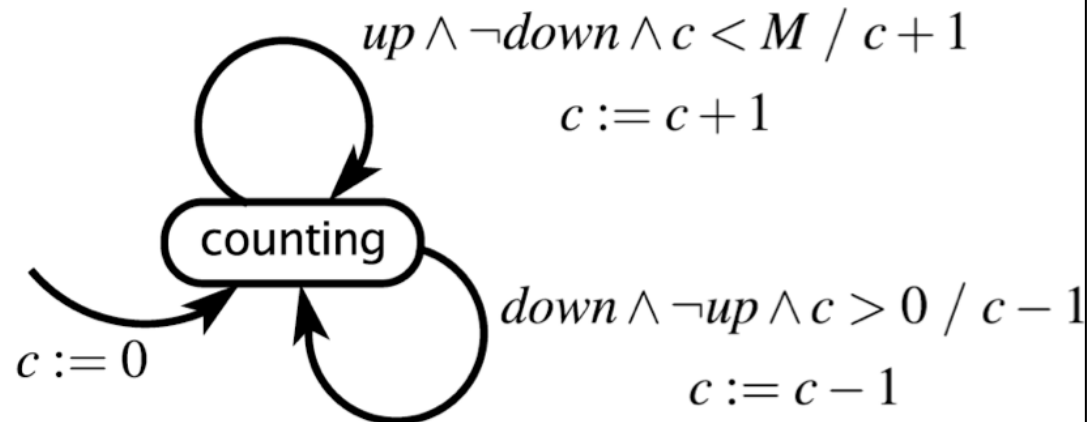
output: $count$: $\{0, \dots, M\}$



variable: c : $\{0, \dots, M\}$

inputs: $up, down$: pure

output: $count$: $\{0, \dots, M\}$



Extended State Machines

- difference from the basic FSM notation
 - ✦ variable declarations are shown explicitly
 - ✦ variables may be initialized in the initial state
 - ✦ transition annotations now have the form
guard / output action
set action

Extended State Machines

- Set action: assignments to variables
 - ◆ These assignments are made after the guard has been evaluated and the outputs have been produced

Extended State Machines in picture

variable declaration(s)
input declaration(s)
output declaration(s)

